MixBytes()

[SECURITY AUDIT REPORT]

Lido Lend Easy Track Factories

PREPARED FOR LIDO LEND

REPORT DATE

November 11, 2026

[CONTENTS]

Table of Contents

1. Introduction 3
   1.1 Disclaimer 3
   1.2 Executive Summary 3
   1.3 Project Overview 5
   1.4 Security Audit Methodology 8
   1.5 Risk Classification 10
   1.6 Summary of Findings 11
2. Findings Report 12
   2.1 Critical 12
   2.2 High 12
   2.3 Medium 12
   2.4 Low 13
      L-1 setMarketSupplyCap is unbounded, and a zero cap means unlimited 13
      L-2 AddGuardian permits an arbitrary quorum change in the same motion 14
      L-3 A motion gated on live state reverts when that state drifts, stalls indefinitely, and becomes enactable by anyone if it drifts back 15
      L-4 Guardian actions are market-scoped in the payload but registry-scoped on-chain 17
      L-5 Batch actions have no maximum length, so an oversized motion can be created but never enacted 18
      L-6 The preseed amount is priced by the payload-supplied vault, so a hostile wrapper spends the Preseeder's whole balance 19
      L-7 A guardian can block AddGuardian and ReplaceGuardian by front-running the motion with setDelegate 20
      L-8 UnbanAccounts silently clears sub-quorum flags and shortens activation delays 22
      L-9 Exit Book vault allow-listing checks only for a non-zero address 23
      L-10 LidoLendActivateMarket accepts any oracle, and a wrong oracle is permanent 24
      L-11 Re-arming a consumed pauser needs a motion, so MIN_PAUSE_DURATION must exceed the motion window, but nothing enforces it 26
3. About MixBytes 27

1.1 Disclaimer

The audit makes no statements or warranties regarding the utility, safety, or security of the code, the suitability of the business model, investment advice, endorsement of the platform or its products, the regulatory regime for the business model, or any other claims about the fitness of the contracts for a particular purpose or their bug-free status.

1.2 Executive Summary

Lido Lend is a fork of Morpho Blue for isolated lending markets, extended with per-market policy controls, collateral maturation, guardian-based screening, and tokenized ERC-4626 lender positions. This engagement covered the seven EasyTrack EVMScript factories that give a trusted committee bounded, delayed governance paths over that protocol: market activation, market policy and fees, IRM configuration, guardian membership, risk stewards, CircuitBreaker wiring, and Exit Book allowlists. The underlying Lido Lend contracts were treated as referenced dependencies and read only as far as needed to validate the factories against them.

The interim security review was conducted over 6 working days by three auditors via manual code review and a proprietary AI-assisted analysis tool.

A factory's security value is entirely in its validation: it decides which payloads a committee may enact and what calls they expand into. The review therefore concentrated on parity between each factory's checks and the rules its target actually enforces, on the correctness of the emitted CallsScript, and on behavior when protocol state moves between motion creation and enactment. Alongside that, we worked through our standard internal checklist covering access control, input validation, encoding and decoding correctness, arithmetic safety, and state-dependent logic. The project-specific vectors examined during this engagement are listed below.

Emitted script correctness. We recomputed every selector, struct layout, and calldata body against the real target signatures, including the head/tail encoding of the one dynamic parameter (`unbanGuardianSignals(bytes32[],address)`). All emitted calls match their targets in selector, parameter order, and encoding. The seven-call activation script is ordered correctly and provably so: `applyIrmConfig` and `setMarketIrm` must precede `createMarket`, because the core's `_createMarket` probes `IIrm.borrowRate`, which resolves through the router to a config that must already exist; `preseedMarket` and `setFee` must follow it. The preseeder's supply to the core is exact against `SharesMathLib` - 1000 asset units for `1e9` shares on an empty market - and the `MAX_FEE`, `MAX_DELAY_DURATION`, and virtual-share constants mirrored locally all match their originals. The deposit into the vault is where the problem is, below.

Enactment re-execution removes the stale-script class. `EasyTrack.enactMotion` re-runs `createEVMScript` and requires the result to hash to the value recorded at creation, so every state-dependent check re-runs at enactment and any drift fails closed rather than executing a stale script. This is load-bearing, and it downgrades a number of otherwise-serious creation-versus-enactment concerns to recoverable liveness issues.

The preseed floor is decimals-blind, and we raise it as an observation rather than a finding. `MIN_VAULT_ASSET_UNITS` is a fixed `1e6` wei, while a genuine wrapper's decimals offset makes `previewMint(1e9)` return 1 wei for every sub-18-decimal token, so the floor is always the binding value and the shares are burned to `address(0xdead)`. The amount burned therefore scales with the loan token's decimals rather than with any intended value: 1 whole token at 6 decimals, ten thousand at 2, and at 18 decimals the deposit is `1e-9` tokens - enough to pass the share-count gate but too small to act as the donation buffer it exists to provide. `_validateActivation` performs no decimals check. Every market currently contemplated uses an 18- or 6-decimal loan token, so the costly end of this range is not reachable in practice today and we did not raise it as a finding; the 18-decimal end, where the buffer is ineffective, applies to the markets actually planned. We recommend scaling the floor by the loan token's decimals and bounding the result, so that neither end depends on the token happening to be well-behaved.

Validation parity is otherwise accurate. The collateral-activation config validation mirrors `CollateralActivationLib.setConfig` one-for-one across all eleven rules, and the settlement, suspect-window, and LLTV checks are exact mirrors of their targets. We confirmed that `setFee` correctly omits an explicit accrual because the core accrues internally before changing the fee, that guardian delegate uniqueness is enforced at the registry in all three directions rather than only at the factory, and that a banned borrower can still withdraw collateral and repay, so bans deny borrowing rather than lock funds. Supply caps are the exception: they are unbounded in both the factory and the controller, and `cap == 0` means unlimited rather than zero - a loosening operation one mistyped digit away.

Risk steward membership has no floor, and an emptied set removes the only path to freezing a market. Neither the factory nor the dashboard bounds the steward set at either end. `RemoveRiskSteward` checks only that the target is currently a steward, and `RiskStewardDashboard.removeMarketSteward` is a bare `EnumerableSet.remove`, so the last steward on a market can be removed and the set left empty. That state is not inert: `LidoLendMarketFactory` grants `FREEZER_ROLE` on each controller to the dashboard alone, and the dashboard exposes `freezeMarket` only behind `onlyMarketSteward`. With zero stewards there is no holder of that capability and no one can freeze the market, while `UNFREEZER_ROLE` sits with `LidoLendOwner` - so the emergency lever is lost through the ordinary removal path, with the tighten-only delay and activating-lock levers going with it. Recovery requires a fresh motion and a full objection period, which is precisely the delay the freeze exists to pre-empt. In the other direction, `AddRiskSteward` performs no market-existence check and appoints into any `bytes32` id; we confirmed this is deliberate rather than an oversight, as the committee is expected to seat stewards before activation and both the factory tests and the dashboard unit tests assert the flow. The consequence is benign - a stale entry becomes live only if a market is later created at that exact id, and ids are content hashes of the market params - but the two behaviors share a root cause: membership is a raw set with no invariant over its size or its relationship to live markets. We recommend enforcing a minimum of one steward on removal, and treating the appointment of a steward to an unknown market id as a state worth surfacing to the committee rather than one that passes silently.

Build inputs are pinned and reproducible. The compiler is fixed at `solc 0.8.6` with `evm_version: berlin`, and every factory in scope declares an exact `pragma solidity 0.8.6` rather than a floating range. OpenZeppelin is pinned to `4.3.2` in three places that agree - the Brownie remapping, the `package.json` alias, and the import paths themselves - and the library is vendored into the repository rather than resolved at build time. Both lockfiles are committed, `eth-brownie` is pinned exactly, and the Python and Node versions are fixed by `.python-version` and `.nvmrc`. We note two things for completeness, neither of which affects build output: the GitHub Actions in the Slither workflow are pinned to floating major tags rather than commit SHAs, and the reformatted `IERC20.sol` will differ byte-for-byte from a freshly installed OpenZeppelin, which is worth knowing during deployment verification so it is not mistaken for a modification.

1.3 Project Overview

Summary

CLIENT	Lido
CATEGORY	Lending
PROJECT	Lido Lend Easy Track Factories
TYPE	Solidity
PLATFORM	EVM
TIMELINE	08.09.2026 - 21.09.2026

Scope of Audit

`LidoLendActivateMarket.sol` `LidoLendCircuitBreakerActions.sol` `LidoLendGuardianActions.sol` `LidoLendMarketManagerActions.sol` `LidoLendRiskStewardActions.sol` `LidoLendApplyIrmConfig.sol` `LidoLendExitBookActions.sol` `LidoLendFactoryBase.sol` `LidoLendOwnerFactoryBase.sol` `LidoLendPreseeder.sol` `ILidoLend.sol`

Versions Log

DATE	COMMIT HASH	NOTE
08.09.2026	81476a8f0e419d0eeefd17c38be875512aa6ed74	Initial Commit
21.09.2026	4caa9117435631ef352d263c901fa9e26cfccfb4	Commit for Re-audit

Mainnet Deployments

FILE	ADDRESS	BLOCKCHAIN
LidoLendPreseeder.sol	0x3F107d1fbCA49F425321Cb27f6Ef5CbB94a72c41	Ethereum
LidoLendActivateMarket.sol	0x17bb2A2B5983D29392A5d31D627D4C8B13383fe6	Ethereum
LidoLendCircuitBreakerActions.sol	0x4e89cc970F0DBAE2f8C38C083D5D345B4F9e76b8	Ethereum
LidoLendGuardianActions.sol	0x9fE41afb2114bF9430D7f9B1Fb65f0a4B9DeF779	Ethereum
LidoLendMarketManagerActions.sol	0x9D75EaE321976e85e472847AFD21246F1f87dC43	Ethereum
LidoLendRiskStewardActions.sol	0x7276CE5f92911216d41C7b0A9C3575DE873763E9	Ethereum
LidoLendApplyIrmConfig.sol	0x5795E73C0c2007ceD7D65cE4FC21c0C09F14C668	Ethereum
LidoLendExitBookActions.sol	0xA30b212c3657fa745feaE4b48B7Eb0A8F429C604	Ethereum

All on-chain checks in this section were performed against Ethereum mainnet state at block `26385136` (block hash: `0xac42f2648cf27ad3a8baf9f513e815baeb4138555047d804f63ac82d73c7b9d5`).

We verified the deployed contracts against the re-audit commit `4caa9117435631ef352d263c901fa9e26cfccfb4`, and they match. All eight contracts were created by the EOA `0x44810Fd195015649d7F884a63926a55F0a538511` in consecutive plain `CREATE` transactions (nonces 63-70), with no proxy involved.

Rebuilding from the audited commit with the project's own compiler settings (`solc 0.8.6`, EVM version `berlin`, optimizer enabled with 200 runs - brownie's default, as `brownie-config.yaml` at this commit has no optimizer section) yields runtime bytecode matching the deployed code. The only differences are the constructor immutables and the IPFS metadata hash.

The immutables read from the deployed code match the constructor arguments recorded in the client's deployment artifacts and the Lido Lend mainnet protocol addresses:

CONTRACT	ADDRESS
Core	0x026a770badc7126c7252Ef8d407E5b9C56147491
LidoLendOwner	0x85378a1073E8f338e11A97E090E47e2F0FdD3D5a
IrmRouter	0x81B7cC9523B30429110282d90db2fa90ff443d91
AdaptiveCurveIrm	0xf6f1565F4fD3d29E6bDAb86764A50BCde381422f
LidoLendMarketFactory	0xBe1C1eaBda3eD8a93D3CE3246cf4EeF8111c76C7
CircuitBreaker	0xd0A81A130a303237d338A0b5835edcB011Ab3afd
RiskStewardDashboard	0xC3C241bC213EF3ba16234A7c689999209cE515D7
Morpho exit book	0xAd28EFa8f2ef56a68b0Eb6836f4aE1d2D6459Ecd
ERC-4626 exit book	0x0d2580e4c9b20507eDFAeFC7777d760F12Ad55D4

`LidoLendActivateMarket.lidoLendPreseeder()` points to the deployed Preseeder. The Preseeder grants `EXECUTOR_ROLE` to the Easy Track `EVMScriptExecutor` `0xFE5986E06210aC1eCC1aDCafc0cc7f8D63B3F977` and `DEFAULT_ADMIN_ROLE` and `ERC20_RECOVERER` to the Lido Agent `0x3e40D73EB977Dc6a537aF587D48316feE66E9C8c`; the deployer holds no role.

The protocol side is wired for the factories: `CircuitBreaker.ADMIN()`, `IrmRouter.manager()` and `AdaptiveCurveIrm.manager()` all return the `EVMScriptExecutor`, which holds `MARKET_CREATOR_ROLE`, `MARKET_MANAGER_ROLE`, `GUARDIAN_MANAGER_ROLE` and `FEE_MANAGER_ROLE` on `LidoLendOwner`, `STEWARD_ADMIN_ROLE` on the `RiskStewardDashboard`, and `ALLOWLIST_MANAGER_ROLE` on both exit books. `CircuitBreaker.MIN_PAUSE_DURATION()` is 432000 seconds (5 days) against an Easy Track `motionDuration` of 259200 seconds (3 days), so the invariant described in finding 11 holds at the time of verification.

The `trustedCaller` of all seven factories is `0x4f29aA4c299e17176b48c41D67ee97aE673d29bC`, a Safe v1.4.1 with a 1-of-1 threshold (a single EOA owner, `0x62c89F5768D0FeC0915fb614C5eDd9fdeaA2A3fd`) and no modules enabled. As every motion through these factories can be created by this one key, we recommend adding signers and raising the threshold before the factories are registered in Easy Track.

At the time of verification none of the seven factories is registered in Easy Track `0xF0211b7660680B49De1A7E9f25C65660F0a13Fea` (`isEVMScriptFactory` returns `false`), so a DAO vote is still required.

1.4 Security Audit Methodology

Stage 1 Interim Audit

01 PROJECT ARCHITECTURE REVIEW

OBJECTIVE: UNDERSTAND THE OVERALL STRUCTURE OF THE PROTOCOL AND IDENTIFY POTENTIAL SECURITY RISKS, INCLUDING THE PRIMITIVES AND ABSTRACTIONS IMPLEMENTED BY THE SYSTEM AND WHERE DESIGN OR INTEGRATION WEAKNESSES COULD BE EXPLOITED.

- Understanding overall system design and contract interactions
- Mapping responsibilities of each component and protocol workflows
- Conducting a thorough line-by-line review of contracts and modules
- Tracing execution paths and mapping state transitions
- Identifying trust boundaries and external integrations
- Forming an independent architectural understanding directly from code before validating documentation
- Running the project test suite to observe behavior and encoded invariants

02 ADVERSARIAL CODE REVIEW

OBJECTIVE: IDENTIFY POTENTIAL VULNERABILITIES SPECIFIC TO THE PROTOCOL ARCHITECTURE, BUSINESS LOGIC, AND ECONOMIC MECHANISMS.

- Analyzing the codebase from an attacker's perspective: what can fail, not only what works
- Searching for edge cases, invalid assumptions, unexpected call sequences, and unsafe state transitions
- Attempting to violate protocol invariants and find where guarantees break
- Analyzing economic attack surfaces: oracle dependencies, share pricing, access control, cross-contract interactions
- Writing targeted tests, modelling adversarial scenarios, mutation testing, fuzzing, and PoC exploits
- Independent exploit-path discovery by each auditor to reduce anchoring bias
- Collaborative pair-auditing of high-risk code led by the Team Lead

03 SYSTEMATIC VULNERABILITY ANALYSIS

OBJECTIVE: APPLY STRUCTURED ANALYSIS AND KNOWN ATTACK PATTERNS TO ENSURE COMPREHENSIVE COVERAGE ACROSS THE CODEBASE.

- Reviewing code against an internally maintained vulnerability checklist derived from past exploits and research
- Analyzing common DeFi attack classes: reentrancy, accounting manipulation, oracle manipulation, privilege escalation, state desynchronization
- Using proprietary AI tooling to surface candidate issues across broader attack vectors
- Manually validating and triaging AI-generated candidates

04 CONSOLIDATION OF AUDITORS' REPORTS

OBJECTIVE: MERGE INTERIM INPUTS FROM ALL AUDITORS INTO A COHERENT REPORT WITH CONSISTENT SEVERITY LEVELS AND REPRODUCIBLE FINDINGS.

- Cross-checking findings across auditors; consolidating and deduplicating issues
- Resolving discrepancies in severity assessments
- Using proprietary AI tooling to assist with report drafting and consistency checks
- Manually reviewing and finalizing the report narrative for client review

Stage 2 Re-audit & Mainnet Verification

RE-AUDIT

OBJECTIVE: VERIFY THAT CLIENT REMEDIATIONS ARE CORRECTLY IMPLEMENTED AND THE UPDATED CODE INTRODUCES NO NEW VULNERABILITIES.

- Confirming each remediation matches the original recommendation
- Documenting rationale for any unresolved findings
- Verifying modified logic and integration points remain secure
- Executing the test suite after remediation, targeting fixed areas
- Running AI tooling on the updated codebase

MAINNET DEPLOYMENT VERIFICATION

OBJECTIVE: FINAL VERIFICATION OF DEPLOYED CONTRACTS AND CONFIGURATION BEFORE ISSUING THE PUBLIC REPORT.

- Verifying deployed bytecode against audited source across networks
- Matching bytecode to the build from the audited commit, identical compiler settings
- Reviewing constructor and initializer arguments
- Verifying proxy order, initialization flow, and admin configuration
- Ensuring implementations aren't left uninitialized or exposed to init front-running

1.5 Risk Classification

Severity Level Matrix

SEVERITY	IMPACT: HIGH	IMPACT: MEDIUM	IMPACT: LOW
LIKELIHOOD: HIGH	CRITICAL	HIGH	MEDIUM
LIKELIHOOD: MEDIUM	HIGH	MEDIUM	LOW
LIKELIHOOD: LOW	MEDIUM	LOW	LOW

IMPACT

HIGH - Theft exceeding 0.5% of the protocol's TVL, partial or complete blocking of funds on the contract without the possibility of withdrawal (>0.5%), or loss of user funds exceeding 1% for users interacting with the protocol.

MEDIUM - Contract lock that can only be resolved through a contract upgrade, one-time theft of rewards or an amount up to 0.5% of the protocol's TVL, or funds lock where withdrawal is still possible by an administrator.

LOW - One-time contract lock that can be resolved by an administrator without requiring a contract upgrade.

LIKELIHOOD

HIGH - An event with an estimated 50–60% probability of occurring within a year, which can be triggered by any actor (e.g. due to a market condition that the actor cannot influence).

MEDIUM - An unlikely event (10–20% probability of occurring) that can be triggered by a trusted actor.

LOW - A highly unlikely event that can only be triggered by the contract owner.

ACTION REQUIRED

CRITICAL - Must be fixed as soon as possible.

HIGH - Strongly advised to be fixed in order to minimize potential risks.

MEDIUM - Recommended to be fixed to improve security and stability.

LOW - Recommended to be fixed to improve overall robustness and efficiency.

FINDING STATUS

● FIXED - The recommended fixes have been implemented and the issue no longer impacts the security of the protocol.

● PARTIALLY FIXED - The fixes have been partially implemented, reducing the impact, but the issue has not been fully resolved.

● ACKNOWLEDGED - The fixes were not implemented; the finding is unresolved or was accepted by the team without code changes.

1.6 Summary of Findings

Findings Count

0	0	0	11	11
CRITICAL	HIGH	MEDIUM	LOW	TOTAL

Findings Statuses

All finding statuses reflect the state of the code at the latest revision listed in the Versions Log.

ID	FINDING	SEVERITY	STATUS
L-1	setMarketSupplyCap is unbounded, and a zero cap means unlimited	LOW	ACKNOWLEDGED
L-2	AddGuardian permits an arbitrary quorum change in the same motion	LOW	ACKNOWLEDGED
L-3	A motion gated on live state reverts when that state drifts, stalls indefinitely, and becomes enactable by anyone if it drifts back	LOW	FIXED
L-4	Guardian actions are market-scoped in the payload but registry-scoped on-chain	LOW	ACKNOWLEDGED
L-5	Batch actions have no maximum length, so an oversized motion can be created but never enacted	LOW	ACKNOWLEDGED
L-6	The preseed amount is priced by the payload-supplied vault, so a hostile wrapper spends the Preseeder's whole balance	LOW	ACKNOWLEDGED
L-7	A guardian can block AddGuardian and ReplaceGuardian by front-running the motion with setDelegate	LOW	ACKNOWLEDGED
L-8	UnbanAccounts silently clears sub-quorum flags and shortens activation delays	LOW	ACKNOWLEDGED
L-9	Exit Book vault allow-listing checks only for a non-zero address	LOW	ACKNOWLEDGED

ID	FINDING	SEVERITY	STATUS
L-10	LidoLendActivateMarket accepts any oracle, and a wrong oracle is permanent	LOW	ACKNOWLEDGED
L-11	Re-arming a consumed pauser needs a motion, so MIN_PAUSE_DURATION must exceed the motion window, but nothing enforces it	LOW	ACKNOWLEDGED

[2 FINDINGS REPORT]

2.1 Critical

NO CRITICAL SEVERITY FINDINGS

2.2 High

NO HIGH SEVERITY FINDINGS

2.3 Medium

NO MEDIUM SEVERITY FINDINGS

2.4 Low

L-1 setMarketSupplyCap is unbounded, and a zero cap means unlimited

LOW

ACKNOWLEDGED

DESCRIPTION

`_createSetSupplyCapEVMScript` emits the call with no constraint on the value (`LidoLendMarketManagerActions.sol:267-282`). The controller's `_setSupplyCap` is a bare assignment and no `MAX_SUPPLY_CAP` exists in the protocol, so the factory is the only place a bound could live.

In the controller `cap == 0` means unlimited: `maxDeposit` returns `type(uint256).max` and `_handleSupply` returns early. This convention is deliberate and documented - the specification states "the cap is an admission limit for lender supply; zero means uncapped", and the controller's own NatSpec says "0 means no cap" - so the concern is not a hidden behavior but an unguarded input. The value is validated nowhere along the path, and zero is both the most natural way to express "stop deposits" and the one value that means the opposite, so a cap intended to halt deposits removes the cap entirely.

Setting a cap below current supply does not lock funds, which the specification also confirms: it is enforced only on the post-deposit total, so withdrawals, repays and liquidations are unaffected and deposits resume once supply falls.

This issue is classified Low severity because a mistyped cap loosens rather than locks and is correctable by another motion (Low impact), and only the trusted committee can set it (Low likelihood).

RECOMMENDATION

> We recommend rejecting a zero cap in the factory. This does not contradict the documented convention: it keeps "uncapped" available at the controller while forcing it through a path where the intent is explicit rather than expressible by a single mistyped digit in a motion payload. A dedicated action, or a distinct sentinel, would make the uncapped case legible during the objection window. We also recommend adding a same-value check, for consistency with the other actions in this factory.

CLIENT'S COMMENTARY

Acknowledged. Zero is considered as a valid supply cap value, explicitly supported by `LidoLendMarketController`.

L-2 AddGuardian permits an arbitrary quorum change in the same motion

LOW

ACKNOWLEDGED

DESCRIPTION

`AddGuardian` takes `newQuorum` as a payload field and emits two calls, `[addGuardian, setGuardiansQuorum]`, so the quorum is not derived from the membership change - it is whatever the motion says. The only constraint is (`LidoLendGuardianActions.sol:192`):

```
require(newQuorum != 0 && newQuorum <= guardianCount + 1, ERROR_INVALID_QUORUM);
```

That is an upper bound against the guardian count after the addition, plus non-zero. It never reads the current quorum, so every value in `[1, guardianCount + 1]` is accepted and a motion titled "add a guardian" can carry a quorum of one - letting a single guardian, or a single delegate key since delegates need not be unique, ban any borrower on every market the registry serves (Low-4). The target adds nothing: `GuardianSignals._setQuorum` rejects only zero and values above the guardian count.

Bundling the two calls is itself necessary for `RemoveGuardian`, which would hit `WouldBreakQuorum` if the quorum were not lowered first, and both orderings are correct - `RemoveGuardian` emits `[setGuardiansQuorum, removeGuardian]`. The add path inherited the same free parameter without needing it, since a quorum that was valid before remains valid against a larger guardian set.

This issue is classified Low severity because a factory should bound what the committee may do relative to a DAO vote, and here it does not (Low impact), while only the trusted committee can file such a motion (Low likelihood).

RECOMMENDATION

> We recommend bounding the quorum delta relative to the current value in the add and remove paths, for instance permitting only the current quorum or a change of one step, and routing deliberate quorum changes through the dedicated `SetGuardiansQuorum` action so the motion's intent is legible. A majority floor on that action is also worth considering.

CLIENT'S COMMENTARY

Acknowledged. The explicit `newQuorum` parameter in both `AddGuardian` and `RemoveGuardian` is intentional. It allows the committee to update membership and set the desired valid quorum atomically, without automatic quorum adjustments or special handling of boundary cases.

L-3 A motion gated on live state reverts when that state drifts, stalls indefinitely, and becomes enactable by anyone if it drifts back

LOW · FIXED

DESCRIPTION

Several factories gate script construction on live protocol state that the motion itself does not control, and `enactMotion` re-runs the factory before executing (`EasyTrack.sol:183`), so each check is evaluated again at enactment:

- `UnfreezeMarkets` requires every listed market to still be frozen (`LidoLendMarketManagerActions.sol:294-297`).
- `AddRiskSteward` requires the steward not to be appointed yet (`LidoLendRiskStewardActions.sol:102`), and `RemoveRiskSteward` the converse (`:120`).

Drift reverts the motion: If the same change is applied out of band during the objection window - the `UNFREEZER_ROLE` holder unfreezes the market, or `STEWARD_ADMIN_ROLE` appoints the steward directly - the motion's own precondition is now false and enactment reverts. For `UnfreezeMarkets` the check is per element in a loop, so a single market unfrozen out of band reverts the entire batch and the remaining markets stay frozen, exactly when governance is trying to restore service. The same loop has no duplicate check, so a batch listing one market twice passes creation and then reverts at enactment, where the re-executed factory sees the first unfreeze already applied. None of this strictness is required by the target: the controller's `unfreeze` is idempotent, a bare `isFrozen = false` with no `NotFrozen` error.

The motion then stalls rather than resolving: A reverted enactment does not clear the motion; it stays in the queue. `duration` is only a minimum wait, never an expiry (`:174`), so nothing ages it out, and it keeps consuming one of the 24 global slots until someone cancels it.

A stalled motion remains a live instruction: `enactMotion` is unrestricted - `whenNotPaused` and the elapsed-duration check are its only guards, and `creator` is read from storage and replayed into the factory, so the factory's `onlyTrustedCaller` is satisfied no matter who submits the transaction. If the state later drifts back - the market is frozen again, or the steward is removed again - the precondition holds once more and any address can enact it. The committee's intent was scoped to the situation at filing time, but the motion outlives that situation and can fire much later, against a state the committee never reviewed, at a moment chosen by an unrelated party.

This issue is classified Low severity because every case fails closed rather than executing anything incorrect, the delayed re-execution reapplies an action the committee did approve, and `cancelMotion` clears a stalled motion at the cost of the elapsed delay (Low impact), and it requires either an operator mistake in composing the batch or an out-of-band role holder moving the same state the motion targets during the objection window (Low likelihood).

RECOMMENDATION

> We recommend making the factory tolerate drift wherever the target action is already idempotent, which removes both the batch-atomicity failure and the stall at once: in `UnfreezeMarkets`, de-duplicate market ids in the loop and treat an already-unfrozen market as a skip rather than a revert, retaining `_getMarketControllerOrRevert` so unknown ids still fail fast. Where a precondition must be kept, we recommend the committee cancel a motion as soon as its subject is handled out of band, rather than leaving it queued.

> More generally, a motion that is stale in this way is indistinguishable on-chain from one that is merely waiting, so we recommend monitoring for motions whose script construction currently reverts. An expiry after which a passed motion can no longer be enacted would bound the window in which a stalled motion can be revived, and is worth considering at the EasyTrack level.

CLIENT'S COMMENTARY

Partially fixed in 4caa911. Removed the `isFrozen()` check from `UnfreezeMarkets` so that a market unfrozen through another path does not block the remaining markets in the batch.

L-4 Guardian actions are market-scoped in the payload but registry-scoped on-chain

LOW

ACKNOWLEDGED

DESCRIPTION

Every guardian action in `LidoLendGuardianActions` takes a `marketId`, resolves the registry through `_guardianSignalsOf(marketId)`, and encodes `abi.encode(marketId, ...)`:

contracts/EVMScriptFactories/lidoLendFactories/LidoLendGuardianActions.sol

But `marketId` is only a lookup key. `LidoLendOwner` resolves it to a registry and then calls a registry-global setter, and in `GuardianSignals` the guardian set, quorum, and suspect window are registry-wide singletons - `_guardians`, `_quorum`, `_suspectWindow` - while only `_signals` is keyed by market. One registry serving several markets is a supported topology: `AddMarket.s.sol` selects a registry by `guardianGroup` and validates reuse through `_validateReusedGuardianSignals`. The specification says so plainly: "Guardian roster, quorum, and suspect-window changes apply to the resolved registry and therefore affect every market sharing it."

So a motion whose payload, decoder output, and calldata all name one market silently reconfigures every market on that registry. Of the six guardian actions, only `UnbanAccounts` is truly market-scoped.

Markets A and B share registry R, five guardians, quorum three. `SetGuardiansQuorum(marketId = A, newQuorum = 1)` is filed to loosen gating on A during an incident. On enactment R's quorum is one for both, so on B a single guardian can ban any borrower unilaterally - and since `_isBanned` triggers at `flaggedBy.length() >= _quorum`, accounts on B already carrying one flag are banned instantly, with no motion naming them. A banned borrower's collateral never matures: `_getUnlockTime` returns `type(uint256).max`.

The objection window is the only control here, and the payload understates what the motion changes.

This issue is classified Low severity because a ban denies borrowing while collateral remains withdrawable and repayable, and the registry-wide setting is correctable by a subsequent motion (Low impact), while only the trusted committee can file the motion and the objection window exposes it before it takes effect (Low likelihood).

RECOMMENDATION

> We recommend making the blast radius explicit in the payload: either take the `guardianSignals` address directly instead of a `marketId`, or require the payload to enumerate every market whose controller resolves to the same registry and verify that list on-chain. Retain `marketId` for `UnbanAccounts`, where it is meaningful.

CLIENT'S COMMENTARY

Acknowledged. The factory follows the existing `LidoLendOwner` interface, where `marketId` identifies the guardian registry to configure. The scope may not be obvious from the payload; the current design is retained, and the associated operational risks are accepted.

L-5 Batch actions have no maximum length, so an oversized motion can be created but never enacted

LOW

ACKNOWLEDGED

DESCRIPTION

`UnbanAccounts`, `UnfreezeMarkets`, and both Exit Book actions require a non-empty array but set no upper limit.

Creation costs less per element than enactment: creation runs the factory's view checks, while enactment re-runs them and then makes one state-changing call per element. So there is a band of batch sizes where `createMotion` succeeds but `enactMotion` cannot fit in a transaction. The motion is then permanently unenactable and can only be cleared by cancelling it, after the committee has waited out the objection period for a result it cannot apply.

A stuck motion also holds its queue slot until someone cancels it, rather than releasing it on enactment. Slot consumption is one per motion and does not depend on the array length, but the queue is shared: `motions` is one global array capped at `MAX_MOTIONS_LIMIT = 24` (`EasyTrack.sol:138`) across every Lido EasyTrack factory, so the cost is borne protocol-wide rather than by Lido Lend alone.

This issue is classified Low severity because the wasted delay and slot are recoverable by `cancelMotion`, which makes no external calls and so always succeeds (Low impact), and all seven factories are `onlyTrustedCaller`, making this an operational mistake rather than an attack primitive (Low likelihood).

RECOMMENDATION

> We recommend adding an explicit maximum batch length to the four batch paths - 32 would match `MAX_GUARDIANS_COUNT` - so that an oversized array is rejected at creation rather than discovered to be unenactable after the objection period has already elapsed.

CLIENT'S COMMENTARY

Acknowledged. Execution gas depends on the action, batch contents, and protocol state, so a fixed batch-size limit would not reliably capture execution costs. Batch sizing remains an operational responsibility; oversized motions can be cancelled and resubmitted as smaller batches.

L-6 The preseed amount is priced by the payload-supplied vault, so a hostile wrapper spends the Preseeder's whole balance

LOW

ACKNOWLEDGED

DESCRIPTION

`previewPreseed` asks the payload-supplied vault what the preseed costs (`LidoLendPreseeder.sol:72-82`). `vaultAssets` is whatever `previewMint(1e9)` returns; `MIN_VAULT_ASSET_UNITS` is a floor and there is no ceiling. Three things then fail to bound the spend:

- The amounts are not in the script. It encodes only `abi.encode(marketParams, preseedVault)` and they are recomputed at execution, so the number reviewed during the objection window is not the number that executes, and `enactMotion`'s hash check cannot bind it.
- `maxDeposit` and `previewDeposit` are calls to that same vault, and `vaultShares >= requiredVaultShares` is satisfied by returning any large number. The only real limit is the Preseeder's whole balance (`:106-115`).
- `validateERC4626Wrapper` checks only getters on the supplied address and never compares `ERC4626_WRAPPER_IMPL`, so a hand-rolled vault passes.

This issue is classified Low severity because the loss is bounded by the Preseeder's own DAO-funded balance and reaches no user deposits or protocol TVL (Low impact), and it requires the trusted committee to file a motion carrying a hostile or wrong vault, which the objection window exposes before enactment (Low likelihood).

RECOMMENDATION

> We recommend bounding the vault leg in `previewPreseed` with `require(vaultAssets <= MIN_DEAD_DEPOSIT_SHARES)` - on an empty vault the share price never exceeds one asset unit per share, so `1e9` is the exact maximum for a canonical vault - and resetting the vault allowance to zero after `deposit`.

CLIENT'S COMMENTARY

Acknowledged. The preseed vault is selected by the trusted committee through an EasyTrack activation motion and is subject to review during the objection period. A motion containing a malicious vault can be rejected or cancelled before execution.

L-7 A guardian can block AddGuardian and ReplaceGuardian by front-running the motion with setDelegate

LOW

ACKNOWLEDGED

DESCRIPTION

`AddGuardian` and `ReplaceGuardian` reject a candidate address that is currently registered as some guardian's delegate, via `_requireNotDelegate` (`LidoLendGuardianActions.sol:328-340`). The check reads live registry state, and `EasyTrack.enactMotion` re-runs `createEVMScript` before executing (`EasyTrack.sol:183`), so it is evaluated again at enactment. Meanwhile `setDelegate` is permissionless for any sitting guardian, takes effect immediately, and does not validate the value (`GuardianSignals.sol:132-138`) - and `LidoLendOwner` exposes no way for governance to override or clear a delegate.

Any single sitting guardian can therefore veto either action, because the motion names its candidate address publicly for the whole objection period. Against `AddGuardian`, the guardian points its own delegate at the pending `guardian` address and enactment reverts with `ACCOUNT_IS_DELEGATE` - the attacker need not be involved in the motion at all, so this blocks the committee from seating any new member, not only from removing an existing one. Against `ReplaceGuardian` the same move works, since the `_allowedGuardian` exemption covers only `oldGuardian`: a guardian other than the one being replaced can occupy the incoming address freely. Two colluding guardians defeat even that exemption: if the committee files `A=>B` and `C=>D`, setting A's delegate to D and C's to B leaves each candidate held by the other attacker-guardian, so neither motion qualifies and both revert.

Each attempt costs a full objection period, and the attacker can re-point delegates for every re-filing, so the block is repeatable rather than one-time. `RemoveGuardian` is unaffected - it performs no delegate check - so a guardian cannot prevent its own eviction this way. But removal shrinks the guardian set rather than rotating it and requires `newQuorum < getGuardianCount()`, so the committee is left evicting members and lowering the quorum with no way to seat replacements.

This issue is classified Low severity because no funds are at risk and the guardian set stays operational, so the cost is a blocked or degraded membership change rather than a loss of the screening function itself (Low impact), and it can only be triggered by a sitting guardian, an actor the protocol already trusts and appoints through a DAO-approved motion (Low likelihood).

RECOMMENDATION

> We recommend removing the guardian's unilateral control over this check rather than relying on the candidate address staying unoccupied. Enforcing delegate uniqueness inside `GuardianSignals.setDelegate` - rejecting a value that is already a guardian or another guardian's delegate - closes the cross-delegation case at the source and makes the factory's check redundant rather than load-bearing.
>
> Failing that, the factory should not let a delegate registration veto a membership change: skip `_requireNotDelegate` for the guardians being removed or replaced in the same motion, and have `replaceGuardian` clear any stale delegate entry pointing at the incoming address. Adding a governance path to clear a delegate would also give the committee a remedy that does not depend on the guardian's cooperation.

CLIENT'S COMMENTARY

Acknowledged. The blocking guardian can be removed through `RemoveGuardian`, which is unaffected by delegate assignments and clears the removed guardian's delegation. The potential griefing is accepted.

L-8 UnbanAccounts silently clears sub-quorum flags and shortens activation delays

LOW

ACKNOWLEDGED

DESCRIPTION

`GuardianSignals.unban` clears the flagger set unconditionally, with no check that the account is banned (`GuardianSignals.sol:322-328`). Clearing is necessary when lifting a real ban, since `_isBanned` re-derives the ban from `flaggedBy.length() >= _quorum`, but it is total rather than bounded to quorum. Neither `LidoLendGuardianActions` nor `LidoLendOwner.unbanGuardianSignals` adds a banned-check, and the latter loops over caller-supplied `marketIds`, so one motion clears flags for an account across many markets.

Applied to an account that was never banned, the call erases whatever flags it had accumulated below quorum. That is not cosmetic: `strikeCount` is the cardinality of `flaggedBy`, and `_getUnlockTime` applies `strikeCount * delayPerStrike` live at read time, so the surcharge disappears from already-in-flight activation lots rather than only from future ones. The only trace is an `Unbanned` event on an account that was never banned - there are no per-flag `Unflagged` events as on the normal retraction path, so indexers tracking flag state will not see the flags disappear.

This issue is classified Low severity because it loosens a delay rather than locking funds and the guardians can re-flag the account (Low impact), and only the trusted committee can file the motion (Low likelihood).

RECOMMENDATION

> We recommend that the factory require each listed account to be effectively banned on each listed market, so that `UnbanAccounts` cannot be used as an undocumented flag reset. If clearing sub-quorum flags is intended, we recommend emitting per-flag events so the state change is observable off-chain.

CLIENT'S COMMENTARY

Acknowledged. Clearing all flags, including sub-quorum flags, is expected behavior of `GuardianSignals.unban()`.

L-9 Exit Book vault allow-listing checks only for a non-zero address

LOW

ACKNOWLEDGED

DESCRIPTION

The two Exit Book actions validate their targets to very different standards. The Morpho path derives each market id and requires the market to exist in the core before allow-listing it (`LidoLendExitBookActions.sol:126-133`). The ERC-4626 path checks only `vaults[i] != address(0)` (`:152-155`), and `setVaultAllowed` on the exit book adds nothing, so any address - an EOA, an unrelated contract, or a typo - can be allow-listed.

The allow-list is a trust boundary whose assumptions the vault itself controls, and the factory is the last place able to constrain it. `ILidoLend.sol` already declares the `LIDO_LEND()` getter that would bind a wrapper to this deployment, so the check is available and simply not made.

This issue is classified Low severity because it costs defence in depth rather than exposing an attack path, and a wrong entry is correctable by another motion (Low impact), and only the trusted committee can allow-list a vault (Low likelihood).

RECOMMENDATION

> We recommend requiring `ILidoLendERC4626(vaults[i]).LIDO_LEND() == lidoLend`, which is the same binding `LidoLendMarketFactory._validateERC4626Wrapper` checks first. This requires `LidoLendExitBookActions` to extend `LidoLendFactoryBase` so the core address is available.

CLIENT'S COMMENTARY

Acknowledged. Vault selection and verification are the trusted committee's responsibility.

L-10 LidoLendActivateMarket accepts any oracle, and a wrong oracle is permanent

LOW

ACKNOWLEDGED

DESCRIPTION

The factory checks only `oracle != address(0)` (`LidoLendActivateMarket.sol:169`) - it does not even require code at the address - and `LidoLendMarketFactory` requires only that it has code, which its own NatSpec states plainly: "Deployment checks only that the address has code" (`LidoLendMarketFactory.sol:446`). No on-chain oracle registry exists anywhere in the protocol. Liquidation is gated solely on `_isHealthy` , which prices raw collateral with `IOracle(oracle).price()` (`LidoLend.sol:723`), and the core applies no sanity bounds to the returned value - no staleness check, no min/max, no deviation cap, not even a non-zero require. A mismatched or manipulable oracle therefore liquidates healthy borrowers or admits undercollateralized debt, and nothing else in the action paths can do so.

Two properties make a mistake here unrecoverable. The oracle is part of `MarketParams` and therefore of the market id preimage, so no setter exists and governance cannot repoint a live market to a corrected oracle - the only remedy is creating a new market and migrating liquidity. And `freeze()` does not stop liquidation: `isFrozen` is consulted only in the borrow and withdraw-collateral hooks, while the liquidate hook reads no freeze state, per its comment that "Liquidation is a forced path." The sole lever that halts oracle-driven liquidations is a blanket `pauseFor` on the controller, which also halts repay, so borrowers cannot restore their own health while it is active.

The scripted market path validates oracles thoroughly in `OraclePreflight` : proxy admin against the Agent, proxy and implementation codehashes against freshly built references, the oracle's own getters against declared config, and `price() != 0` (`OraclePreflight.sol:46-94`). None of it runs on the EasyTrack path.

This issue is classified Low severity because the trusted caller supplies the address and a wrong oracle affects one market (Low impact and Low likelihood). We note it sits at the top of that range: the impact is unbounded within the affected market and, unlike every other finding here, cannot be corrected by a later motion. The residual risk is that an oracle proxy administered by the Agent is indistinguishable from a canonical one during the objection window, and this is the highest-impact unverified address in the activation payload.

RECOMMENDATION

`OraclePreflight` itself cannot be ported into the factory: its functions are not `view` , because they deploy reference contracts to compare codehashes, and they read expected values from TOML config that has no on-chain equivalent. We therefore recommend adding the subset that is expressible in a `view` factory, in descending order of value:

- Require the oracle proxy's admin to equal the Agent. `LidoLendMarketFactory` already exposes `ADMIN` as a public immutable - the same address `OraclePreflight` passes as `expectedAdmin` - and `OssifiableProxy.proxy__getAdmin()` is `external view` , so this check needs no new infrastructure and directly rejects an attacker-administered oracle. It should be a `staticcall` with graceful failure, or proxied oracles should be mandated, since a non-proxy oracle does not expose the getter.
- Require `oracle.code.length != 0` and `IOracle(oracle).price() != 0` . Both are trivial, and the latter catches an uninitialized or broken oracle that would otherwise only be discovered by the

first liquidation.
- Optionally compare `proxy__getImplementation()`'s codehash against expected values baked into the factory as immutables, and assert the collateral binding for known oracle types (`WSTETH() == collateralToken`). Both are worth having, but they are brittle across oracle upgrades and require a type-switch over the two existing implementations.

CLIENT'S COMMENTARY

Acknowledged. Oracle selection and verification are the trusted committee's responsibility.

L-11 Re-arming a consumed pauser needs a motion, so MIN_PAUSE_DURATION must exceed the motion window, but nothing enforces it

LOW

ACKNOWLEDGED

DESCRIPTION

A CircuitBreaker pause is single-use: `pause()` clears the pauser via `registry.setPauser(_pausable, address(0))` (`CircuitBreaker.sol:266-279`), and an expired pauser cannot heartbeat itself back, since `heartbeat` requires `isPauserLive(msg.sender)` (`:253-258`). `registerPauser` is `onlyAdmin` and is the only re-registration path, so re-arming runs through a `RegisterPauser` motion - the sole re-arm action in `LidoLendCircuitBreakerActions` - and that motion must complete inside the pause it is meant to outlast.

The two values live in different systems and nothing relates them. `min_pause_duration = 432000` (5 days) is an immutable of the breaker's deployment (`lido-lend.toml:3`), while `motionDuration` is an EasyTrack admin setting bounded only from below - `_setMotionDuration` enforces `>= MIN_MOTION_DURATION` of 48 hours and imposes no ceiling (`MotionSettings.sol:92-96`). The invariant holds today at 5 days against a mainnet `motionDuration` of 72 hours, but no document, deploy check, or test records it: `deploy_lido_lend_factories.py` reads neither value. Raising `motionDuration` past the minimum pause is a valid admin action that would silently leave a market unpaused before its replacement pauser exists.

There is no fallback pauser on this deployment. `LidoLendMarketFactory` grants controller `PAUSER_ROLE` to the CircuitBreaker alone - its NatSpec calls it the "Sole initial holder of every deployed controller's `PAUSER_ROLE`" (`:264,492`) - and `RESUMER_ROLE` starts with no holder at all. Once the pause lapses with the pauser consumed and the motion still in its objection window, nothing can re-pause the market until that motion enacts.

This issue is classified Low severity because the invariant currently holds with a comfortable margin and the gap is a temporary loss of the pause capability rather than a loss of funds (Low impact), and it requires an administrative change to `motionDuration` on a system whose operators are unlikely to connect it to a breaker parameter (Low likelihood).

RECOMMENDATION

> We recommend documenting the invariant and asserting it where both values are reachable. `MIN_PAUSE_DURATION()` is already declared in `ILidoLend.sol`, so a deployment-time check in `scripts/deploy_lido_lend_factories.py` can require `circuitBreaker.MIN_PAUSE_DURATION() > easyTrack.motionDuration()` with a margin covering enactment latency. Because `motionDuration` can be raised after deployment, we also recommend treating it as a standing operational constraint - checked whenever either value changes, rather than only at deploy time.

CLIENT'S COMMENTARY

Acknowledged. Coordinating pause duration with the EasyTrack motion window remains an operational responsibility.

[3 ABOUT MIXBYTES]

Built on Trust

MixBytes is a blockchain security firm specializing in the analysis of decentralized protocols and smart contract systems. The company helps Web3 teams build resilient protocol architecture, smart contract logic, and economic mechanisms through a combination of AI-assisted analysis and senior human expertise.

Rather than focusing solely on one-time audits before deployment, MixBytes works with protocols across their entire lifecycle - from early architecture design to production audits and ongoing protocol evolution. Our team consists of experienced security researchers, engineers, and protocol analysts with deep expertise in DeFi systems, adversarial protocol analysis, and smart contract security.

Over the years, MixBytes has worked with many of the most widely used protocols in the ecosystem, including Lido, Aave, Curve, 1inch, OKX, Mantle, Fluid, Gearbox, Resolv, and others. To enhance analysis efficiency and coverage, MixBytes also develops and uses internal AI-assisted tooling that helps surface potential risk signals during development and code review.

SECURED TVL	AUDITS DELIVERED	WEB3 PROTOCOLS
$50B+	300+	80+

Protocol Security Lifecycle

MixBytes supports protocols across the full lifecycle of their development and operation.

Web-3 Design Review

Independent assessment of protocol architecture and economic design before critical decisions are embedded in production code.

AI Tooling

AI-assisted analysis integrated into development workflows to surface potential risk signals during protocol development.

Smart Contract Audit

Comprehensive manual verification of smart contract logic and protocol invariants before production deployment.

Security Support

Continuous expert support for protocol upgrades, integrations, governance changes, and evolving attack surfaces after launch.

CONTACTS

hello@mixbytes.io